Gobierno de Reconciliación y Unidad Nacional — El Pueblo, Presidente!

NICARAGUA | Centro Nacional de Innovación y Tecnología Francisco "El Chele" Moreno

INATEC Tecnológico Nacional

MINED Un Ministerio en la Comunidad

SETEC Secretaría Técnica para Atención a las Universidades

<hackathon/> Nicaragua | KRONOX 2026

10 años ¡Siempre más allá!

Lectura complementaria

Python desde cero

Fundamentos para crear soluciones tecnológicas.
Todo lo esencial del taller, explicado con calma para leer en casa.

📅 8 de octubre de 2026 | 📍 CI Nicaragua | 🐍 Nivel: desde cero



Autor

Marvin Rafael Meza Pineda

Centro Nacional de Innovación y Tecnología
Francisco "El Chele" Moreno (CI Nicaragua)





Antes de empezar

Hoy diste tus primeros pasos en la programación. Escribiste un programa que conversa con el usuario, toma decisiones, repite tareas, hace cálculos y hasta modela los productos de una tienda. Este documento existe para que puedas **volver a todo eso con calma**, sin el reloj del taller encima.

No es un manual para memorizar. Es una lectura para **entender**: cada capítulo explica una idea, por qué existe y cómo se usa, con ejemplos tomados del mismo programa que construimos juntos, `taller.py`. Puedes leerlo de corrido o saltar al tema que más te interese.

> 🧠 **Una idea para llevarte**
> Comprender la lógica de programación es más importante que memorizar código. Los programadores con experiencia buscan la sintaxis en internet todos los días; lo que no se busca es la forma de pensar.

Cómo aprovechar esta lectura

- **Ten Python a mano.** Cada vez que veas un ejemplo, escríbelo y ejecútalo. Leer código ayuda; escribirlo enseña.
- **Cambia los ejemplos.** ¿Qué pasa si cambias un número, borras una línea o escribes algo mal a propósito? Romper el código es una excelente forma de entenderlo.
- **Al final** encontrarás un glosario y las referencias bibliográficas para seguir aprendiendo.
- **Busca los recuadros.** 🧠 explican conceptos, 💡 dan consejos, ⚠️ advierten errores comunes y ✅ te proponen practicar.

Contenido

1. ¿Qué es programar?
2. Python: el lenguaje
3. Cómo se ejecuta un programa
4. Variables y tipos de datos
5. Operadores
6. Tomar decisiones
7. Repetir tareas
8. Funciones
9. Clases y objetos
10. Todo junto: taller.py
11. Buenas prácticas
12. ¿Y ahora qué sigue?

Capítulo 1

¿Qué es programar?

Antes de escribir una sola línea de Python, conviene entender qué estamos haciendo cuando programamos.



Una computadora es extraordinariamente rápida, pero no sabe hacer nada por sí sola: necesita que alguien le diga, **paso a paso**, qué hacer. **Programar es escribir esas instrucciones** de forma clara, ordenada y sin ambigüedades, para resolver un problema.

A una secuencia de pasos que resuelve un problema se le llama **algoritmo**. La palabra suena técnica, pero usamos algoritmos todo el tiempo: una receta de cocina, las instrucciones para armar un mueble o la ruta que sigues para llegar a tu casa son algoritmos.

🍳 Un algoritmo humano

1. Calentar el sartén.
2. Agregar aceite.
3. Romper el huevo y ponerlo en el sartén.
4. **Si** te gusta con sal, agregar sal.
5. Esperar **hasta que** la clara esté blanca.

🐍 El mismo pensamiento en código

Fíjate en las palabras **si** y **hasta que** de la receta. En programación se convierten en **condicionales** (`if`) y **bucles** (`while`). Toda la lógica de un programa se construye con unas pocas ideas como estas, combinadas de muchas formas.

La diferencia entre una persona y una computadora es que la persona interpreta. Si la receta dice “un poco de sal”, entiendes qué hacer. La computadora no: necesita instrucciones exactas. Por eso, aprender a programar es, sobre todo, **aprender a pensar con precisión**: dividir un problema grande en pasos pequeños y claros.

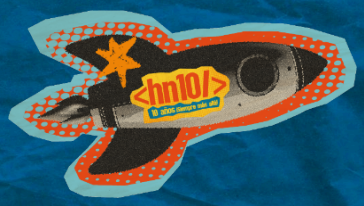
🧠 **La programación está en todas partes**

El cajero automático, el semáforo, la aplicación de mensajes de tu celular y el sistema que calcula tu factura siguen instrucciones que alguien escribió. Cualquier persona, sin importar su perfil, puede aprender a escribirlas.

Capítulo 2

Python: el lenguaje

Por qué elegimos Python para empezar, de dónde viene y dónde se usa hoy.



Las computadoras solo entienden ceros y unos. Escribir así sería imposible para una persona, por eso existen los **lenguajes de programación**: formas de escribir instrucciones que las personas podemos leer y que luego se traducen a algo que la máquina ejecuta.

1 Un poco de historia

Python fue creado por el programador neerlandés **Guido van Rossum**, quien empezó a desarrollarlo a finales de 1989 y publicó la primera versión en **1991**. Su objetivo era un lenguaje fácil de leer y agradable de escribir. El nombre no viene de la serpiente, sino del grupo de comedia británico **Monty Python**, del que van Rossum era fanático.

Hoy usamos **Python 3**. La versión anterior, Python 2, dejó de recibir soporte en 2020, así que cualquier material que encuentres debería ser de Python 3.

2 Por qué es ideal para empezar

Compara cómo se muestra un saludo en pantalla en Python y en Java, otro lenguaje muy popular:

Python

```
print("¡Hola, mundo!")
```

Java

```
class Hola {
  public static void main(String[] a) {
    System.out.println("¡Hola, mundo!");
  }
}
```

Ambos programas hacen lo mismo. Python usa palabras comunes y casi ningún símbolo extraño, así que puedes concentrarte en **la lógica** en lugar de pelear con la sintaxis. Esa filosofía está escrita en un pequeño texto llamado **El Zen de Python**; puedes leerlo completo escribiendo `import this` en Python. Dos de sus principios resumen el espíritu del lenguaje:

“La legibilidad cuenta.” · “Lo simple es mejor que lo complejo.”

3 Sus características principales

Característica	Qué significa para ti
Interpretado	Escribes el código y lo ejecutas de inmediato, sin pasos intermedios. Si hay un error, lo ves enseguida.
Tipado dinámico	No tienes que declarar si una variable guarda texto o números: Python lo detecta solo.
Multiparadigma	Permite programar paso a paso, con funciones o con clases y objetos, según lo que necesites.
Gran comunidad	Millones de personas lo usan, así que casi cualquier duda ya tiene respuesta y casi cualquier problema ya tiene una librería que ayuda.

4 Dónde se usa

🌐 Desarrollo web

Servidores y APIs con frameworks como Django, Flask o FastAPI.

🤖 Inteligencia artificial y datos

Machine Learning, análisis de datos y modelos predictivos.

⚙️ Automatización

Reportes, tareas repetitivas de oficina, bots y extracción de datos de la web.

🔐 Ciberseguridad

Scripts para pruebas de seguridad, auditorías y administración de redes.

Capítulo 3

Cómo se ejecuta un programa

Qué pasa cuando presionas ▷ y cómo leer los mensajes de error sin miedo.



Un programa de Python es un archivo de texto con extensión `.py`. Cuando lo ejecutas, el **intérprete de Python** lo lee **de arriba hacia abajo, línea por línea**, y va haciendo lo que cada línea indica. Ese orden importa: si usas una variable antes de crearla, Python no sabrá qué es.

Hay dos formas de ejecutar `taller.py` en VS Code: el botón ▷ de la esquina superior derecha, o escribir en la terminal:

```
Terminal
$ python taller.py
```

> 💡 **Si la terminal no reconoce "python"**
> En Windows prueba con `py taller.py`; en macOS y Linux, con `python3 taller.py`. Si nada funciona, reinstala Python y marca la opción **"Add python.exe to PATH"**.

1 Los errores son tus aliados

Tarde o temprano verás un mensaje rojo como este. Es normal: los programadores profesionales ven errores todos los días. La clave es **leerlos**, porque casi siempre dicen exactamente qué pasó y dónde.

```
Terminal
Traceback (most recent call last):
  File "taller.py", line 14, in <module>
    edad = int(edad_str)
ValueError: invalid literal for int() with base 10: 'diecisiete'
```

- **Empieza por la última línea:** dice el **tipo de error** (`ValueError`) y su causa: se intentó convertir el texto *"diecisiete"* en número.
- **Luego mira la línea indicada:** *line 14* te lleva directo al lugar del problema.

Error	Qué suele significar
`SyntaxError`	Algo está mal escrito: faltan dos puntos `:`, un paréntesis o unas comillas.
`IndentationError`	Los espacios al inicio de una línea no son correctos (lo verás en el capítulo 6).

Error	Qué suele significar
`NameError`	Usaste un nombre que no existe: quizá está mal escrito o lo usaste antes de crearlo.
`TypeError`	Mezclaste tipos de datos que no se pueden combinar, como sumar texto con un número.
`ValueError`	El tipo es correcto, pero el valor no sirve, como convertir *“hola”* en número.

Capítulo 4

Variables y tipos de datos

Cómo guarda la información un programa y por qué no todos los datos son iguales.



1 Variables: cajas con nombre

Una **variable** es como una caja con una etiqueta: guarda un dato para usarlo después. Se crea con el signo `=`, que en programación no significa "igual" sino **"guarda esto aquí"**.

```
nombre = "Ana"        # guarda el texto "Ana" en la caja llamada nombre
edad = 17             # guarda el número 17 en la caja llamada edad
edad = edad + 1       # toma el valor actual (17), le suma 1 y lo vuelve a guardar: 18
```

Los nombres de variables deben ser descriptivos. En Python se escriben en minúsculas y con guiones bajos entre palabras (estilo *snake_case*): `nombre_participante` es mucho más claro que `np` o `x`.

⚠ **Reglas para los nombres**

No pueden empezar con un número, no pueden tener espacios y distinguen mayúsculas de minúsculas: `edad` y `Edad` son dos variables diferentes.

2 Los tipos de datos básicos

Tipo	Nombre	Ejemplos	Para qué sirve
`str`	Texto (*string*)	`"Ana"`, `"PROD-01"`	Nombres, mensajes, códigos
`int`	Número entero	`17`, `5`, `-3`	Edades, cantidades, stock
`float`	Número decimal	`650.0`, `0.15`	Precios, porcentajes
`bool`	Verdadero o falso	`True`, `False`	Respuestas de sí o no

Puedes preguntarle a Python de qué tipo es un dato con `type()`: por ejemplo, `type(17)` responde `<class 'int'>`.

3 Conversar con el usuario: input()

La función `input()` muestra una pregunta, espera a que el usuario escriba y devuelve lo escrito. Hay un detalle fundamental: **siempre devuelve texto**, aunque el usuario escriba un número.

```
edad_str = input("¿Qué edad tienes? ")   # si escribes 17, guarda el TEXTO "17"
edad = int(edad_str)                     # int() lo convierte en el NÚMERO 17
proxima_edad = edad + 1                  # ahora sí podemos sumar: 18
```

🧠 **¿Por qué hay que convertir?**
Para Python, `"17"` (texto) y `17` (número) son cosas distintas. Sumar `"17" + 1` produce un `TypeError`. Las funciones `int()`, `float()` y `str()` convierten un dato de un tipo a otro.

4 Mostrar resultados: print() y las f-strings

`print()` muestra información en pantalla. Para combinar texto con variables, la forma más clara es una **f-string**: se escribe una `f` antes de las comillas y las variables van entre llaves.

```
print(f"¡Hola, {nombre}! El próximo año cumplirás {proxima_edad} años.")
```

Resultado

```
¡Hola, Ana! El próximo año cumplirás 18 años.
```

✅ **Pruébalo**
Pide al usuario su ciudad y su comida favorita, y muéstralas en una sola frase con una f-string. Después, quita la `f` y observa qué pasa.

Capítulo 5

Operadores

Los símbolos con los que el programa calcula y compara.

1 Aritméticos: para calcular

Operador	Operación	Ejemplo	Resultado
`+`	Suma	`7 + 2`	`9`
`-`	Resta	`7 - 2`	`5`
`*`	Multiplicación	`7 * 2`	`14`
`/`	División (siempre da decimal)	`7 / 2`	`3.5`
`//`	División entera (sin decimales)	`7 // 2`	`3`
`%`	Residuo de la división	`7 % 2`	`1`
`**`	Potencia	`2 ** 3`	`8`

💡 **Un truco con %**

Si `numero % 2` da `0`, el número es par. Es una de las formas más usadas de este operador.

2 De comparación: para preguntar

Comparan dos valores y responden con un `bool`: `True` o `False`. Son la base de las decisiones del siguiente capítulo.

Operador	Pregunta	Ejemplo	Resultado
`==`	¿Son iguales?	`5 == 5`	`True`
`!=`	¿Son distintos?	`5 != 3`	`True`
`<` · `>`	¿Menor? ¿Mayor?	`2 > 9`	`False`
`<=` · `>=`	¿Menor o igual? ¿Mayor o igual?	`2 <= 5`	`True`

⚠️ **= no es lo mismo que ==**

Un solo `=` **guarda** un valor en una variable. Dos `==` **preguntan** si dos valores son iguales. Confundirlos es uno de los errores más comunes al empezar.

3 Lógicos: para combinar preguntas

Operador	Significado	Ejemplo
`and`	Las dos condiciones deben cumplirse	`edad >= 15 and edad <= 18`
`or`	Basta con que se cumpla una	`respuesta == "si" or respuesta == "sí"`
`not`	Invierte el resultado	`not hay_stock`

4 Atajos útiles

Cuando quieres actualizar una variable con su propio valor, Python ofrece una forma corta. La usamos en la clase `Producto` para descontar el stock:

```
stock = stock - 2     # forma larga
stock -= 2            # forma corta, hace exactamente lo mismo (también existen +=, *= y /=)
```

Capítulo 6

Tomar decisiones

Cómo hace un programa para elegir qué camino seguir.



Hasta ahora, los programas hacían siempre lo mismo. Las **estructuras condicionales** permiten que hagan cosas distintas según la situación, igual que tú decides llevar paraguas solo **si** está lloviendo.

1 if y else

```
nivel_experiencia = input("¿Tienes experiencia previa? (si/no): ").lower()

if nivel_experiencia == "si":
    print("-> ¡Excelente! Tienes bases para retos más avanzados.")
else:
    print("-> Perfecto. Este taller desde cero te dará bases sólidas.")
```

Se lee casi como español: *si la respuesta es "si", muestra el primer mensaje; si no, muestra el segundo.* Solo uno de los dos caminos se ejecuta, nunca ambos.

🧠 **¿Para qué sirve .lower()?**
Convierte el texto a minúsculas. Así, si el usuario escribe *SI*, *Si* o *si*, la comparación funciona igual. Es un pequeño detalle que hace el programa más amable.

2 La indentación: el orden visual que Python exige

Fíjate en los **4 espacios** al inicio de las líneas con `print`. Se llaman **indentación** y le dicen a Python qué líneas pertenecen al `if` y cuáles al `else`. En otros lenguajes es una cuestión de estilo; en Python es **obligatoria**.

⚠️ **Error frecuente: olvidar los dos puntos**
Toda línea con `if`, `else`, `for`, `while`, `def` o `class` termina en `:`. Sin ellos aparece un `SyntaxError`.

💡 **VS Code te ayuda**
Al presionar **Enter** después de los dos puntos, el editor agrega los 4 espacios por ti.

3 Más de dos caminos: elif

Cuando hay más de dos posibilidades, se usa `elif` ("si no, si..."). Python revisa las condiciones en orden y ejecuta **solo la primera** que se cumpla:

```
if nota >= 90:
    print("Excelente")
elif nota >= 70:
    print("Aprobado")
else:
    print("Necesitas reforzar")
```

✅ **Pruébalo**

Modifica el programa del taller para que responda distinto a “si”, a “no” y a cualquier otra respuesta, usando `elif`.

Capítulo 7

Repetir tareas

Los bucles: la razón por la que las computadoras nunca se cansan.



Imagina tener que escribir `print` mil veces. Los **bucles** permiten repetir un bloque de código tantas veces como haga falta, y son una de las herramientas que más tiempo ahorran en programación.

1 El bucle for

Se usa cuando sabes **cuántas veces** quieres repetir algo, o cuando quieres recorrer una colección de elementos.

```
for modulo in range(1, 4):
    print(f">> Cargando módulo {modulo} de programación...")
```

Resultado

```
>> Cargando módulo 1 de programación...
>> Cargando módulo 2 de programación...
>> Cargando módulo 3 de programación...
```

En cada vuelta, la variable `modulo` toma un valor nuevo. `range(1, 4)` genera los números **desde el 1 hasta antes del 4**: es decir, 1, 2 y 3.

Forma	Genera
`range(5)`	0, 1, 2, 3, 4 (empieza en 0 si no indicas inicio)
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8 (el tercer número es el salto)
`range(3, 0, -1)`	3, 2, 1 (cuenta hacia atrás)

2 El bucle while

Se usa cuando no sabes cuántas veces repetirás algo, sino que quieres seguir **mientras** se cumpla una condición:

```
respuesta = ""
while respuesta != "salir":
    respuesta = input("Escribe algo (o 'salir' para terminar): ")
    print(f"Escribiste: {respuesta}")
```

⚠️ **Cuidado con el bucle infinito**

Si la condición de un `while` nunca deja de cumplirse, el programa se repetirá para siempre. Si te pasa, detenlo en la terminal con `Ctrl` + `C` y revisa que algo dentro del bucle cambie la condición.

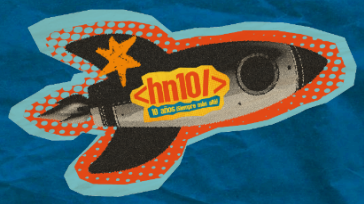
✅ **Pruébalo**

Usa un `for` para mostrar la tabla de multiplicar del 7, del 7 × 1 al 7 × 10.

Capítulo 8

Funciones

Cómo empaquetar una tarea para usarla una y otra vez.



Una **función** es un bloque de código con nombre que realiza una tarea específica. Piénsala como una **máquina**: le das algo, lo procesa y te devuelve un resultado. Ya usaste varias funciones que trae Python, como `print()`, `input()` e `int()`. Ahora puedes crear las tuyas.

```
def calcular_precio_con_iva(precio_base):
    """Calcula el precio final aplicando un impuesto del 15%"""
    iva = precio_base * 0.15
    total = precio_base + iva
    return total

precio_final = calcular_precio_con_iva(100.0)
print(f"Precio con IVA (15%): ${precio_final}")    # Precio con IVA (15%): $115.0
```

1 Las partes de una función

Parte	Qué hace
`def`	Le avisa a Python que vas a **definir** una función.
`calcular_precio_con_iva`	El nombre: debe describir lo que hace, idealmente con un verbo.
`(precio_base)`	El **parámetro**: el dato que la función necesita para trabajar.
`"""..."""`	El **docstring**: una descripción de la función para quien la lea.
`return total`	Entrega el resultado a quien llamó la función.

2 ¿Por qué usar funciones?

- **Reutilizar:** escribes el cálculo una vez y lo usas con 1 producto o con 1000.
- **Organizar:** divides un problema grande en tareas pequeñas con nombres claros.
- **Corregir en un solo lugar:** si el IVA cambia, modificas una línea y todo el programa queda actualizado.

🧠 **Definir no es ejecutar**
Escribir `def` solo crea la función; no hace nada hasta que la **llamas** por su nombre con paréntesis: `calcular_precio_con_iva(100.0)`.

✅ **Pruébalo**

Crea una función `aplicar_descuento(precio, porcentaje)` que devuelva el precio con descuento. Pruébala con `aplicar_descuento(200, 10)` ; el resultado debería ser `180.0` .

Capítulo 9

Clases y objetos

La programación orientada a objetos: modelar el mundo real dentro del código.



Cuando un programa crece, conviene agrupar los datos y las acciones que van juntos. Una tienda tiene productos; cada producto tiene código, nombre, precio y stock, y se puede vender. La **programación orientada a objetos (POO)** permite representar esas "cosas" del mundo real tal como las pensamos.

Clase

El molde o plano

Describe qué datos y qué acciones tendrá cada elemento. Es como el plano de una casa: no es una casa, pero dice cómo construirla.

Objeto

El elemento real

Se crea a partir de la clase y tiene sus propios datos. Con un mismo plano se pueden construir muchas casas distintas.

1 Crear la clase

```
class Producto:
    def __init__(self, codigo, nombre, precio, stock):
        self.codigo = codigo
        self.nombre = nombre
        self.precio = precio
        self.stock = stock

    def mostrar_informacion(self):
        print(f"[{self.codigo}] {self.nombre} | ${self.precio} | Stock: {self.stock}")

    def realizar_venta(self, cantidad_comprada):
        if cantidad_comprada <= self.stock:
            self.stock -= cantidad_comprada
            print(f"-> ¡Venta exitosa! Se vendieron {cantidad_comprada} de {self.nombre}.")
        else:
            print(f"-> ¡Error! No hay suficiente stock de {self.nombre}.")
```

Concepto	Qué significa
Atributos	Los datos de cada objeto: `codigo`, `nombre`, `precio` y `stock`.
Constructor `__init__`	Un método especial que se ejecuta **automáticamente** al crear un objeto y le asigna sus datos iniciales.

Concepto	Qué significa
`self`	Significa “este objeto en particular”. Gracias a `self`, cada producto guarda sus propios datos sin mezclarse con los de otros.
Métodos	Funciones que pertenecen a la clase: las acciones que el objeto sabe hacer.

2 Crear objetos y usarlos

```
producto1 = Producto("PROD-01", "Laptop Core i5", 650.0, 5)
producto2 = Producto("PROD-02", "Teclado Mecánico RGB", 45.0, 10)

producto1.realizar_venta(2)       # el stock de producto1 baja de 5 a 3
producto1.mostrar_informacion()   # [PROD-01] Laptop Core i5 | $650.0 | Stock: 3
producto2.mostrar_informacion()   # [PROD-02] Teclado Mecánico RGB | $45.0 | Stock: 10
```

Observa lo importante: al vender laptops, **solo cambia el stock de `producto1`**. Cada objeto tiene su propio **estado**. Esa es la gran ventaja de la POO: puedes tener cientos de productos, cada uno con sus datos, todos creados con un mismo molde.

✅ **Pruébalo**

Agrega a la clase un método `reponer_stock(cantidad)` que sume unidades al inventario. Luego intenta vender 20 teclados y lee el mensaje que aparece.

Capítulo 10

Todo junto: taller.py

Cómo las piezas de cada capítulo forman un programa completo.



El programa que construimos en el taller recorre, en orden, todas las ideas de esta lectura. Mirarlo de principio a fin ayuda a ver cómo cada concepto se apoya en el anterior:

Bloque	Qué hace el programa	Conceptos que usa
1	Saluda y calcula tu edad del próximo año	Variables, `input()`, `int()`, operadores, f-strings
2	Responde según tu experiencia y "carga módulos"	`if` / `else`, `.lower()`, `for`, `range()`
3	Calcula un precio con IVA	Funciones, parámetros, `return`
4	Define cómo es un producto	Clases, constructor, `self`, métodos
5	Crea productos y realiza una venta	Objetos, llamadas a métodos, estado

Al ejecutarlo con las respuestas *Ana*, *17* y *no*, este es el resultado:

```
python taller.py

==================================================
   BIENVENIDOS AL TALLER DE PYTHON - CI NICARAGUA
==================================================
¿Cuál es tu nombre? Ana
¡Hola, Ana! Comencemos a programar.
¿Qué edad tienes? 17
El próximo año cumplirás 18 años.

--- 2. CONTROL DE FLUJO Y LOGICA ---
¿Tienes experiencia previa? (si/no): no
-> Perfecto. Este taller desde cero te dará bases sólidas.

Iniciando sistema de carga de módulos:
>> Cargando módulo 1 de programación...
>> Cargando módulo 2 de programación...
>> Cargando módulo 3 de programación...
¡Sistema listo y operativo! 🚀

--- 3. FUNCIONES ---
Precio base: $100.0 | Precio con IVA (15%): $115.0

--- 4. MODELANDO ENTIDADES CON CLASES Y OBJETOS ---

--- Creando Instancias (Objetos) de nuestra Entidad ---
[PROD-01] Producto: Laptop Core i5 | Precio: $650.0 | Stock disponible: 5
[PROD-02] Producto: Teclado Mecánico RGB | Precio: $45.0 | Stock disponible: 10

--- Ejecutando una transacción comercial ---
-> ¡Venta exitosa! Se vendieron 2 unidad(es) de Laptop Core i5.

--- Estado actualizado de la Entidad después de la venta ---
[PROD-01] Producto: Laptop Core i5 | Precio: $650.0 | Stock disponible: 3
```

🚀 **Un paso más: proteger el programa**

Si alguien escribe *diecisiete* en la edad, el programa se detiene con un `ValueError`. Con `try` y `except` puedes atrapar ese error y responder con amabilidad:

```
try:
    edad = int(input("¿Qué edad tienes? "))
except ValueError:
    print("Por favor, escribe tu edad con números.")
```

Capítulo 11

Buenas prácticas

Hábitos que distinguen el código de un aficionado del de un profesional.



El código se escribe una vez, pero se lee muchas veces: por tus compañeros, por quien lo mantenga después y por ti mismo dentro de unos meses. Por eso la comunidad de Python acordó una guía de estilo llamada **PEP 8**. Estos son sus consejos más importantes para empezar:

✅ Hazlo así	❌ Evita esto
`precio_final = 115.0`	`pf = 115.0` (nombres que nadie entiende)
Nombres de variables y funciones en `snake_case`	`PrecioFinal` o `preciofinal` para variables
Nombres de clases en `CamelCase`: `Producto`	`producto` o `PRODUCTO` para una clase
4 espacios por nivel de indentación	Mezclar espacios y tabuladores
Comentarios que explican el **porqué**	Comentarios que repiten lo que el código ya dice
Funciones cortas que hacen una sola cosa	Una función gigante que lo hace todo

Hábitos de aprendizaje

- **Practica un poco cada día.** Veinte minutos diarios valen más que cinco horas una vez al mes.
- **Lee los errores con calma.** La última línea casi siempre te dice qué pasó.
- **Escribe el código, no lo copies.** Tus dedos también aprenden.
- **Divide los problemas.** Si algo parece imposible, resuelve primero una parte pequeña.
- **Explica tu código en voz alta.** Si puedes explicarlo, lo entiendes; si te trabas, ahí está la duda.
- **Pregunta y comparte.** La comunidad de programación es enorme y generosa.

Capítulo 12

¿Y ahora qué sigue?

Un mapa para continuar tu camino después del taller.



Lo que aprendiste hoy es la base sobre la que se construye todo lo demás. Estos son los siguientes pasos naturales, en un orden sugerido:

#	Tema	Para qué te sirve
1	**Listas y diccionarios**	Guardar muchos datos juntos, como todos los productos de una tienda.
2	**Manejo de errores** (`try` / `except`)	Hacer programas que no se rompan ante datos inesperados.
3	**Archivos**	Guardar y leer información para que no se pierda al cerrar el programa.
4	**Módulos y librerías**	Usar código que otras personas ya escribieron, con `import` y `pip` .
5	**Git y GitHub**	Guardar la historia de tu código y compartirlo. ¡Es el taller de esta tarde!
6	**Un área de especialización**	Desarrollo web (Django, Flask, FastAPI), ciencia de datos, automatización o inteligencia artificial.

🧠 **Aprende construyendo**

La mejor forma de avanzar es crear proyectos pequeños que te interesen: una calculadora de notas, un control de gastos, un juego de adivinar números o un inventario para un negocio de tu familia. Cada proyecto te obligará a aprender algo nuevo.

Glosario

Término	Definición
Algoritmo	Secuencia ordenada de pasos para resolver un problema.
Argumento	El valor concreto que le pasas a una función al llamarla.
Atributo	Un dato que pertenece a un objeto, como su precio o su stock.
Bucle	Estructura que repite un bloque de código (`for` , `while`).
Clase	Molde que define los datos y las acciones de un tipo de objeto.
Condicional	Estructura que elige qué código ejecutar (`if` , `elif` , `else`).
Constructor	Método `__init__` que prepara un objeto al crearlo.
Función	Bloque de código con nombre que realiza una tarea y puede devolver un resultado.
Indentación	Espacios al inicio de una línea que indican a qué bloque pertenece.

Término	Definición
Intérprete	Programa que lee y ejecuta el código de Python.
Método	Función que pertenece a una clase; una acción que sabe hacer un objeto.
Objeto (instancia)	Elemento concreto creado a partir de una clase, con sus propios datos.
Parámetro	Nombre que recibe un dato dentro de la definición de una función.
Tipo de dato	La clase de información que guarda un valor: `str`, `int`, `float`, `bool`.
Variable	Nombre que guarda un dato para usarlo después.

Referencias bibliográficas

Fuentes consultadas y recomendadas para profundizar. Las marcadas como *acceso libre* se pueden leer gratis en línea.

Downey, A. B. (2015). *Think Python: How to think like a computer scientist* (2.ª ed.). O'Reilly Media. *(acceso libre)* https://greenteapress.com/wp/think-python-2e/

Guo, P. J. (2013). Online Python Tutor: Embeddable web-based program visualization for CS education. En *Proceedings of the 44th ACM Technical Symposium on Computer Science Education* (pp. 579–584). ACM. https://doi.org/10.1145/2445196.2445368

Lutz, M. (2013). *Learning Python* (5.ª ed.). O'Reilly Media.

Matthes, E. (2023). *Python crash course: A hands-on, project-based introduction to programming* (3.ª ed.). No Starch Press.

Microsoft. (s. f.). *Python in Visual Studio Code*. *(acceso libre)* https://code.visualstudio.com/docs/languages/python

Peters, T. (2004). *PEP 20 – The Zen of Python*. Python Software Foundation. *(acceso libre)* https://peps.python.org/pep-0020/

Python Software Foundation. (s. f.). *El tutorial de Python*. Documentación de Python 3. *(acceso libre)* https://docs.python.org/es/3/tutorial/

Python Software Foundation. (s. f.). *La biblioteca estándar de Python*. Documentación de Python 3. *(acceso libre)* https://docs.python.org/es/3/library/

Python Software Foundation. (s. f.). *Preguntas frecuentes generales sobre Python*. *(acceso libre)* https://docs.python.org/es/3/faq/general.html

Sweigart, A. (2019). *Automate the boring stuff with Python: Practical programming for total beginners* (2.ª ed.). No Starch Press. *(acceso libre)* https://automatetheboringstuff.com/

van Rossum, G., Warsaw, B., y Coghlan, N. (2001). *PEP 8 – Style guide for Python code*. Python Software Foundation. *(acceso libre)* https://peps.python.org/pep-0008/

Herramientas para practicar

👀 Python Tutor

`pythontutor.com`

Ejecuta tu código paso a paso y mira cómo cambian las variables. Ideal para entender bucles y objetos.

📂 Repositorio del taller

`github.com/Meza2805/python-desde-cero-kronox-2026`

El código completo, la guía de cada bloque y los retos para seguir practicando.

¡Gracias por programar conmigo!

Si este taller te gustó y te ayudó a dar tus primeros pasos, me encantaría seguir en contacto contigo en GitHub.

⭐ **Sígueme en GitHub**

`github.com/Meza2805`

🌟 **Deja una estrella al repositorio**

`python-desde-cero-kronox-2026`

Una estrella ⭐ es la forma de decir “esto me sirvió” en GitHub, y ayuda a que más estudiantes encuentren este material.

Marvin Rafael Meza Pineda
Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno



